Assignment Description

This project simulates a small **Library Management System** that manages books, patrons, and transactions (borrowing and returning). It demonstrates **OOP design** and **advanced C++ features** in an integrated, real-world context.

1 Readme Documentation

```
Usage

On startup the program loads the data files and shows the menu:


===== Library Menu =====
1. View all books
2. View all patrons
3. Check out a book
4. Return a book
5. Add a book
6. Add a patron
7. Search by author
8. Search by genre
0. Save and exit


Enter the number of an option, then answer its prompts. Titles,
author names, genres, and book types must be typed exactly as
stored (they are case-sensitive).

Sample session:


Choice: 3
Patron ID: 2
Book title: Frankenstein
```

```
Book Frankenstein checked out by Patron ID 2. Due date:
10/31/2026

Choice: 3
Patron ID: 1
Book title: Frankenstein
Error: Book Frankenstein is already checked out.

Choice: 4
Patron ID: 2
Book title: Frankenstein
Book Frankenstein returned by Patron ID 2.

Choice: 8
Genre: Science
Books in genre Science:
Title: Quantum Physics 101
Author: Albert Einstein
Genre: Science
Status: Available
File Size: 4.5 MB
...

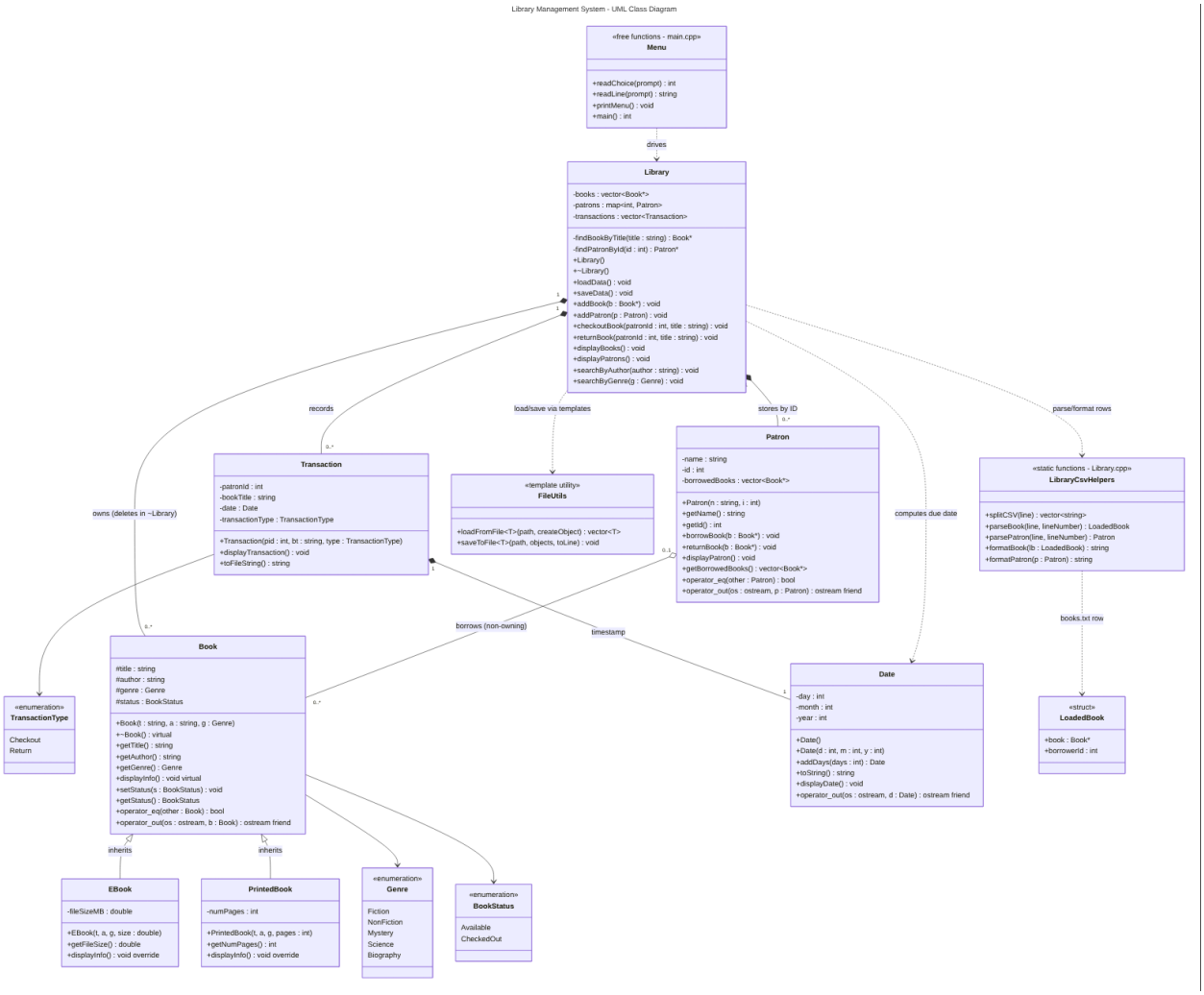
Choice: 0
Data saved. Goodbye!
```

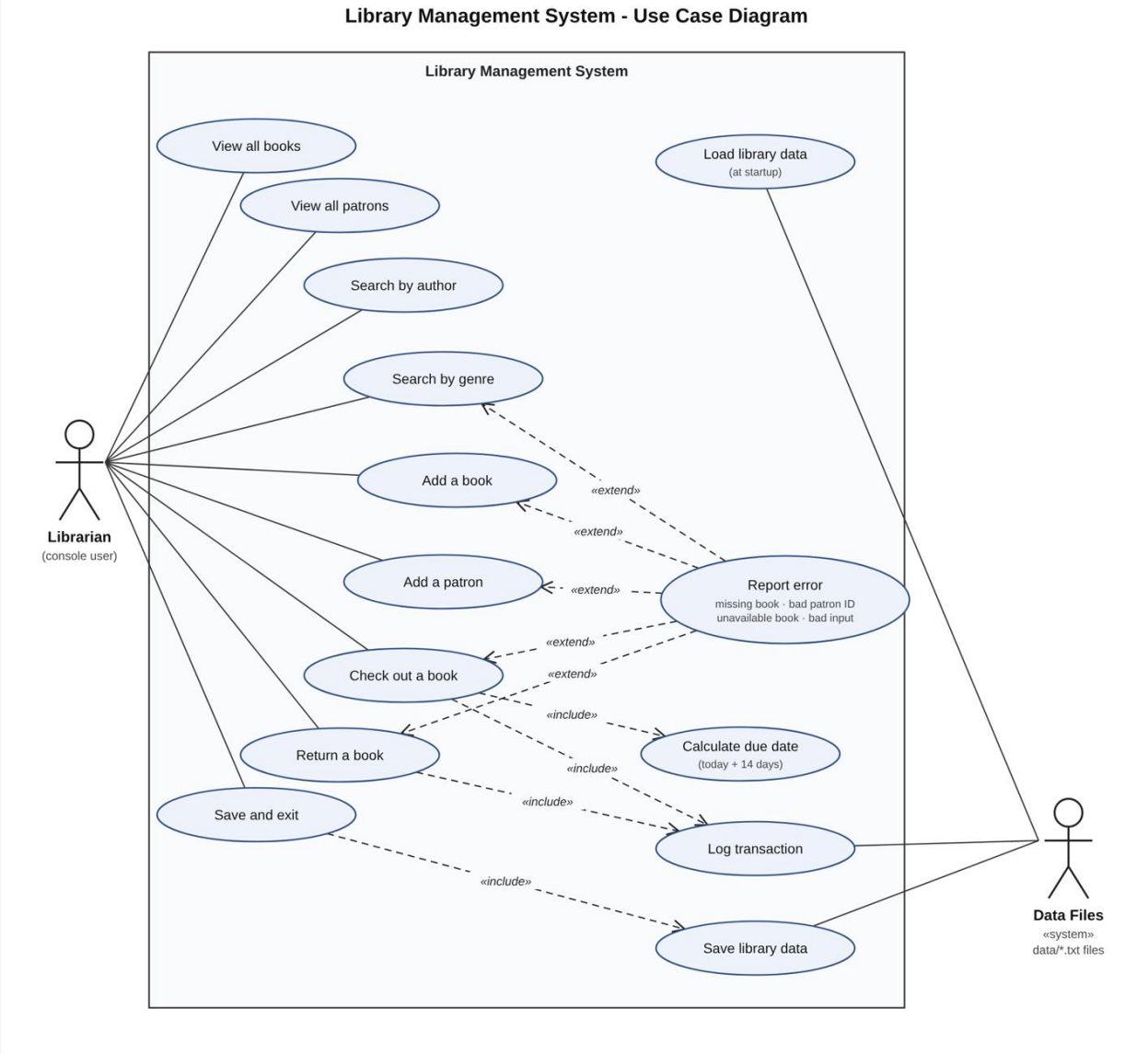
2 Flowchart Screen Shots or Pseudocode

```
-- Project Structure

LibraryManagementSystem/
    ├── data/
    │   ├── books.txt           - Book records (CSV)
    │   ├── patrons.txt         - Patron records (CSV)
    │   └── transactions.txt    - Append-only transaction log
    ├── include/
    │   ├── Book.h  EBook.h  PrintedBook.h
    │   ├── Patron.h
    │   ├── Transaction.h  Date.h
    │   ├── Library.h
    │   ├── Enums.h
    │   └── FileUtils.h         - Template load/save helpers
    └── src/
        ├── Book.cpp  EBook.cpp  PrintedBook.cpp
        ├── Patron.cpp
        ├── Transaction.cpp  Date.cpp
        ├── Library.cpp
        ├── Enums.cpp
        └── main.cpp            - Menu loop and input helpers
```

3 UML and Use Case Diagrams




Library Management System - Use Case Diagram
Library Management System
View all books
View all patrons
Search by author
Search by genre
Add a book
Add a patron
Check out a book
Return a book
Save and exit
Load library data
(at startup)
Report error
missing book · bad patron ID
unavailable book · bad input
Calculate due date
(today + 14 days)
Log transaction
Save library data
«extend»
«extend»
«extend»
«extend»
«extend»
«include»
«include»
«include»
«include»
Librarian
(console user)
Data Files
«system»
data/*.txt files

4 Source Code of All files (.h, .cpp)

```
#ifndef BOOK_H
#define BOOK_H

#include <string>
#include <iostream>

#include "Enums.h"

class Book {

    protected:
        std::string title;
        std::string author;
        Genre genre;
        BookStatus status;

    public:
        std::string getTitle() const;
        std::string getAuthor() const;
        Genre getGenre() const;

        Book(std::string t, std::string a, Genre g);
        virtual ~Book() = default;

        virtual void displayInfo() const;
        void setStatus(BookStatus s);
        BookStatus getStatus() const;

        bool operator==(const Book& other) const;
        friend std::ostream& operator<<(std::ostream& os, const
Book& b);
};
#endif // BOOK_H
```

```h
#ifndef DATE_H
#define DATE_H

#include <string>
#include <iostream>
#include <ctime>

class Date {
    int day;
    int month;
    int year;

public:
    Date();
    Date(int d, int m, int y);

    Date addDays(int days) const;
    std::string toString() const;

    void displayDate() const;

    friend std::ostream& operator<<(std::ostream& os, const
Date& date);
};

#endif // DATE_H
```

```h
#ifndef EBOOK_H
#define EBOOK_H

#include <string>

#include "Book.h"

class EBook : public Book {
    double fileSizeMB;
public:
    EBook(std::string t, std::string a, Genre g, double size);
    double getFileSize() const;
    void displayInfo() const override;
};

#endif // EBOOK_H
```

```h
#ifndef ENUMS_H
#define ENUMS_H

#include <string>

enum class Genre {
    Fiction,
    NonFiction,
    Mystery,
    Science,
    Biography
    };

enum class BookStatus {
    Available,
    CheckedOut
    };

enum class TransactionType {
    Checkout,
    Return
    };

std::string genreToString(Genre g);
Genre stringToGenre(const std::string& str);
std::string statusToString(BookStatus s);
std::string transactionTypeToString(TransactionType t);

#endif // ENUMS_H
```

```cpp
#ifndef FILEUTILS_H
#define FILEUTILS_H

#include <string>
#include <vector>
#include <fstream>
#include <stdexcept>


template <typename T>
std::vector<T> loadFromFile(const std::string& path,
                            T (*createObject)(const
std::string& line, int lineNumber)) {
    std::ifstream file(path);
    if (!file) {
        throw std::runtime_error("Could not open file: " +
path);
    }

    std::vector<T> objects;
    std::string line;
    int lineNumber = 0;
    while (std::getline(file, line)) {
        lineNumber++;
        if (line.empty()) continue; // Skip empty lines
        try {
            objects.push_back(createObject(line, lineNumber));
        } catch (const std::exception& e) {
            throw std::runtime_error("Error in file " + path +
" at line " + std::to_string(lineNumber) + ": " + e.what());
        }
    }
    return objects;
}

template <typename T>
```

```
void saveToFile(const std::string& path, const std::vector<T>&
objects,
                std::string (*toLine)(const T& obj)) {
    std::ofstream file(path);
    if (!file) {
        throw std::runtime_error("Could not open file for
writing: " + path);
    }

    for (const T& obj : objects) {
        file << toLine(obj) << "\n";
    }
    file.close();
}

#endif // FILEUTILS_H
```

```h
#ifndef LIBRARY_H
#define LIBRARY_H

#include <string>
#include <vector>
#include <map>
#include <iostream>
#include <sstream>
#include <stdexcept>

#include "Book.h"
#include "Patron.h"
#include "Transaction.h"
#include "FileUtils.h"
#include "EBook.h"
#include "PrintedBook.h"

class Library {

    std::vector<Book*> books;
    std::map<int, Patron> patrons;
    std::vector<Transaction> transactions;

    Book* findBookByTitle(const std::string& title) const;
    Patron* findPatronById(int id);

    public:
        Library() = default;
        ~Library();

        Library(const Library&) = delete;
        Library& operator=(const Library&) = delete;

        void loadData();
        void saveData();
        void addBook(Book* b);
```

```
        void addPatron(const Patron& p);
        void checkoutBook(int patronId, std::string title);
        void returnBook(int patronId, std::string title);
        void displayBooks() const;
        void displayPatrons() const;
        void searchByAuthor(const std::string& author) const;
        void searchByGenre(Genre g) const;
};

#endif // LIBRARY_H
```

```h
#ifndef PATRON_H
#define PATRON_H

#include <string>
#include <vector>
#include <iostream>
#include <algorithm>

#include "Book.h"


class Patron {

    std::string name;
    int id;
    std::vector<Book*> borrowedBooks;

    public:
        Patron(std::string n, int i);
        std::string getName() const;
        int getId() const;
        void borrowBook(Book* b);
        void returnBook(Book* b);
        void displayPatron() const;

        bool operator==(const Patron& other) const;
        friend std::ostream& operator<<(std::ostream& os, const
Patron& p);
        const std::vector<Book*>& getBorrowedBooks() const;
};

#endif // PATRON_H
```

```
#ifndef TRANSACTION_H
#define TRANSACTION_H

#include <string>
#include "Date.h"
#include "Enums.h"

class Transaction {
    int patronId;
    std::string bookTitle;
    Date date;
    TransactionType transactionType;
    public:
        Transaction(int pid, std::string bt, TransactionType
type);
        void displayTransaction() const;
        std::string toFileString() const;
};

#endif // TRANSACTION_H
```

Book.ccp

```cpp
#include "Book.h"

Book::Book(std::string t, std::string a, Genre g)
    : title(t), author(a), genre(g),
status(BookStatus::Available) {}


std::string Book::getTitle() const {
    return title;
}

std::string Book::getAuthor() const {
    return author;
}

Genre Book::getGenre() const {
    return genre;
}

void Book::setStatus(BookStatus s) {
    status = s;
}

BookStatus Book::getStatus() const {
    return status;
}

void Book::displayInfo() const {
    std::cout << "Title: " << title << std::endl;
    std::cout << "Author: " << author << std::endl;
    std::cout << "Genre: " << genreToString(genre) <<
std::endl;
    std::cout << "Status: " << statusToString(status) <<
std::endl;
```

```
}

bool Book::operator==(const Book& other) const {
    return title == other.title && author == other.author;
}

std::ostream& operator<<(std::ostream& os, const Book& b) {
    b.displayInfo();
    return os;
}
```

Date.ccp

```cpp
#include "Date.h"
#include <ctime>
#include <iomanip>
#include <sstream>


Date::Date() {
    std::time_t t = std::time(nullptr);
    std::tm* now = std::localtime(&t);

    day = now->tm_mday;
    month = now->tm_mon + 1; // tm_mon is 0-based
    year = now->tm_year + 1900; // tm_year is years since 1900

}

Date::Date(int d, int m, int y) : day(d), month(m), year(y) {}

Date Date::addDays(int days) const {
    std::tm t{};
    t.tm_mday = day + days;
    t.tm_mon = month - 1; // tm_mon is 0-based
    t.tm_year = year - 1900; // tm_year is years since 1900
    t.tm_hour = 12;

    std::mktime(&t); // Normalize the time structure

    return Date(t.tm_mday, t.tm_mon + 1, t.tm_year + 1900);
}

std::string Date::toString() const {
    std::ostringstream out;

    out << std::setw(2) << std::setfill('0') << month << "/"
        << std::setw(2) << std::setfill('0') << day << "/"
```

```
        << year;
    return out.str();
}

void Date::displayDate() const {
    std::cout << toString() << std::endl;
}

std::ostream& operator<<(std::ostream& os, const Date& date) {
    os << date.toString();
    return os;
}
```

EBook.cpp

```cpp
#include "EBook.h"

EBook::EBook(std::string t, std::string a, Genre g, double
size)
    : Book(t, a, g), fileSizeMB(size) {}

double EBook::getFileSize() const {
    return fileSizeMB;
}

void EBook::displayInfo() const {
    Book::displayInfo();
    std::cout << "File Size: " << fileSizeMB << " MB" <<
std::endl;
}
```

PrintedBook.cpp

```cpp
#include "PrintedBook.h"

PrintedBook::PrintedBook(std::string t, std::string a, Genre g,
int pages)
    : Book(t, a, g), numPages(pages) {}

int PrintedBook::getNumPages() const {
    return numPages;
}

void PrintedBook::displayInfo() const {
    Book::displayInfo();
    std::cout << "Number of Pages: " << numPages << std::endl;
}
```

Enums.cpp

```cpp
#include "Enums.h"
#include <stdexcept>

std::string genreToString(Genre g) {
    switch (g) {

        case Genre::Fiction: return "Fiction";
        case Genre::NonFiction: return "Non-Fiction";
        case Genre::Mystery: return "Mystery";
        case Genre::Science: return "Science";
        case Genre::Biography: return "Biography";
        default: return "Unknown";

    }
}

Genre stringToGenre(const std::string& str) {

    if (str == "Fiction") {
        return Genre::Fiction;

    } else if (str == "Non-Fiction") {
        return Genre::NonFiction;

    } else if (str == "Mystery") {
        return Genre::Mystery;

    } else if (str == "Science") {
        return Genre::Science;

    } else if (str == "Biography") {
        return Genre::Biography;

    } else {
```

```
        throw std::invalid_argument("Invalid genre string: " +
str);

    }
}

std::string statusToString(BookStatus s) {
    switch (s) {
        case BookStatus::Available: return "Available";
        case BookStatus::CheckedOut: return "Checked Out";
        default: return "Unknown";
    }
}

std::string transactionTypeToString(TransactionType t) {
    switch (t) {
        case TransactionType::Checkout: return "Checkout";
        case TransactionType::Return: return "Return";
        default: return "Unknown";
    }
}
```

Library.cpp

```
#include "Library.h"
#include <fstream>

// Helper function to split a CSV line into fields
static std::vector<std::string> splitCSV(const std::string&
line) {
    std::vector<std::string> fields;
    std::stringstream ss(line);
    std::string field;
    while (std::getline(ss, field, ',')) {
        fields.push_back(field);
    }
    return fields;
}

struct LoadedBook {
    Book* book;
    int borrowerId; // 0 if not borrowed
};

// Helper function to parse a book from a CSV line
static LoadedBook parseBook(const std::string& line, int) {
    std::vector<std::string> f = splitCSV(line);
    if (f.size() != 5 && f.size() != 6) {
        throw std::runtime_error("expected 5 or 6 fields, got "
+ std::to_string(f.size()));
    }
    Genre genre = stringToGenre(f[0]);
    Book* book = nullptr;
    if (f[3] == "Printed") {
        book = new PrintedBook(f[1], f[2], genre,
std::stoi(f[4]));
    } else if (f[3] == "EBook") {
        book = new EBook(f[1], f[2], genre, std::stod(f[4]));
```

```
    } else {
        throw std::runtime_error("unknown book type: " + f[3]);
    }
    int borrower = (f.size() == 6) ? std::stoi(f[5]) : 0;
    return LoadedBook{book, borrower};
}

static Patron parsePatron(const std::string& line, int) {
    std::vector<std::string> f = splitCSV(line);
    if (f.size() != 2) {
        throw std::runtime_error("expected 2 fields, got " +
std::to_string(f.size()));
    }
    return Patron(f[1], std::stoi(f[0]));
}

static std::string formatBook(const LoadedBook& lb) {
    const Book* book = lb.book;
    std::string line = genreToString(book->getGenre()) + "," +
book->getTitle() + "," + book->getAuthor() + ",";

    if (const EBook* e = dynamic_cast<const EBook*>(book)) {
        std::ostringstream size;
        size << e->getFileSize();
        line += "EBook," + size.str();
    } else if (const PrintedBook* p = dynamic_cast<const
PrintedBook*>(book)) {
        line += "Printed," + std::to_string(p->getNumPages());
    } else {
        throw std::runtime_error("Unknown book type for
formatting.");
    }

    if (lb.borrowerId != 0) {
        line += "," + std::to_string(lb.borrowerId);
    }
```

```
    return line;
}

static std::string formatPatron(const Patron& p) {
    return std::to_string(p.getId()) + "," + p.getName();
}

Library::~Library() {
    for (Book* book : books) {
        delete book;
    }
}


void Library::loadData() {
    std::vector<Patron> loadedPatrons =
loadFromFile<Patron>("data/patrons.txt", parsePatron);
    for (const Patron& p : loadedPatrons) {
        addPatron(p);
    }

    std::vector<LoadedBook> loadedBooks =
loadFromFile<LoadedBook>("data/books.txt", parseBook);
    for (const LoadedBook& lb : loadedBooks) {
        addBook(lb.book);
        if (lb.borrowerId != 0) {
           findPatronById(lb.borrowerId)->borrowBook(lb.book);
           lb.book->setStatus(BookStatus::CheckedOut);
        }
    }
}

void Library::addBook(Book* b) {
    books.push_back(b);
}
```

```
void Library::addPatron(const Patron& p) {
    if (patrons.count(p.getId()) > 0) {
        throw std::runtime_error("Patron with ID " +
std::to_string(p.getId()) + " already exists.");
    }
    patrons.insert({p.getId(), p});
}

Book* Library::findBookByTitle(const std::string& title) const
{
    for (Book* book : books) {
        if (book->getTitle() == title) {
            return book;
        }
    }
    throw std::runtime_error("Book with title '" + title + "'
not found.");
}

Patron* Library::findPatronById(int id) {
    auto it = patrons.find(id);
    if (it == patrons.end()) {
        throw std::invalid_argument("Patron with ID " +
std::to_string(id) + " not found.");
    }
    return &(it->second);
}

void Library::displayBooks() const {
    std::cout << "====== Books(" << books.size() << ") ======"
<< std::endl;
    for (const Book* book : books) {
        std::cout << *book << std::endl;
    }
}
```

```
void Library::displayPatrons() const {
    std::cout << "====== Patrons(" << patrons.size() << ")
======" << std::endl;
    for (const auto& pair : patrons) {
        std::cout << pair.second << std::endl;
    }
}

void Library::checkoutBook(int patronId, std::string title) {
    Patron* patron = findPatronById(patronId);
    Book* book = findBookByTitle(title);

    if (book->getStatus() == BookStatus::CheckedOut) {
        throw std::runtime_error("Book '" + title + "' is
already checked out.");
    }

    patron->borrowBook(book);
    book->setStatus(BookStatus::CheckedOut);
    transactions.emplace_back(patronId, title,
TransactionType::Checkout);
    Date dueDate = Date().addDays(14); // Assuming a 2-week
checkout period
    std::ofstream log("data/transactions.txt", std::ios::app);
    log << transactions.back().toFileString() << std::endl;
    log.close();
    std::cout << "Book '" << title << "' checked out by Patron
ID " << patronId << ". Due date: " << dueDate << std::endl;
}

void Library::returnBook(int patronId, std::string title) {
    Patron* patron = findPatronById(patronId);
    Book* book = findBookByTitle(title);

    if (book->getStatus() == BookStatus::Available) {
```

```
        throw std::runtime_error("Book '" + title + "' is not
checked out.");
    }

    patron->returnBook(book);
    book->setStatus(BookStatus::Available);
    transactions.emplace_back(patronId, title,
TransactionType::Return);
    std::ofstream log("data/transactions.txt", std::ios::app);
    log << transactions.back().toFileString() << std::endl;
    std::cout << "Book '" << title << "' returned by Patron ID
" << patronId << "." << std::endl;
}

void Library::saveData() {
    std::map<Book*, int> borrowerOf;
    for (const auto& pair : patrons) {
        const Patron& patron = pair.second;
        for (Book* book : patron.getBorrowedBooks()) {
            borrowerOf[book] = patron.getId();
        }
    }

    std::vector<LoadedBook> bookRows;
    for (Book* book : books) {
        int borrowerId = (borrowerOf.count(book) > 0) ?
borrowerOf[book] : 0;
        bookRows.push_back(LoadedBook{book, borrowerId});
    }

    std::vector<Patron> patronRows;
    for (const auto& pair : patrons) {
        patronRows.push_back(pair.second);
    }
```

```
    saveToFile<LoadedBook>("data/books.txt", bookRows,
formatBook);
    saveToFile<Patron>("data/patrons.txt", patronRows,
formatPatron);
}
void Library::searchByAuthor(const std::string& author) const {
    std::cout << "Books by " << author << ":" << std::endl;
    bool found = false;
    for (const Book* book : books) {
        if (book->getAuthor() == author) {
            std::cout << *book << std::endl;
            found = true;
        }
    }
    if (!found) {
        std::cout << "No books found by " << author << "." <<
std::endl;
    }
}

void Library::searchByGenre(Genre g) const {
    std::cout << "Books in genre " << genreToString(g) << ":"
<< std::endl;
    bool found = false;
    for (const Book* book : books) {
        if (book->getGenre() == g) {
            std::cout << *book << std::endl;
            found = true;
        }
    }
    if (!found) {
        std::cout << "No books found in genre " <<
genreToString(g) << "." << std::endl;
    }
}
```

Patron.cpp

```cpp
#include "Patron.h"
#include <ostream>
#include <stdexcept>

Patron::Patron(std::string n, int i) : name(n), id(i) {}

std::string Patron::getName() const {
    return name;
}

int Patron::getId() const {
    return id;
}

void Patron::borrowBook(Book* b) {
    borrowedBooks.push_back(b);
}

void Patron::returnBook(Book* b) {
    auto it = std::find(borrowedBooks.begin(),
borrowedBooks.end(), b);
    if (it != borrowedBooks.end()) {
        borrowedBooks.erase(it);
    } else {
        throw std::invalid_argument(name + " does not have \""
+ b->getTitle() + "\" checked out.");
    }
}

void Patron::displayPatron() const {
    std::cout << "Patron Name: " << name << ", ID: " << id <<
std::endl;
    std::cout << "Borrowed Books: ";
```

```
    if (borrowedBooks.empty()) {
        std::cout << "None";
    } else {
        for (size_t i = 0; i < borrowedBooks.size(); ++i) {
            if (i > 0) {
                std::cout << ", ";
            }
            std::cout << borrowedBooks[i]->getTitle();
        }
    }
    std::cout << std::endl;
}

bool Patron::operator==(const Patron& other) const {
    return id == other.id;
}

std::ostream& operator<<(std::ostream& os, const Patron& p) {
    p.displayPatron();
    return os;
}

const std::vector<Book*>& Patron::getBorrowedBooks() const {
    return borrowedBooks;
}
```

Transaction.cpp

```cpp
#include "Transaction.h"
#include <iostream>
#include <sstream>

Transaction::Transaction(int pid, std::string bt,
TransactionType type)
    : patronId(pid), bookTitle(bt), transactionType(type) {}

void Transaction::displayTransaction() const {
    std::cout << date << " | "
              << transactionTypeToString(transactionType) <<
" | "
              << "Patron ID: " << patronId << " | "
              << "Book Title: " << bookTitle << std::endl;
}

std::string Transaction::toFileString() const {
    std::ostringstream out;
    out << date << ","
        << transactionTypeToString(transactionType) << ","
        << patronId << ","
        << bookTitle;
    return out.str();
}
```

Main.cpp

```cpp
#include "Library.h"

#include <limits>
#include <stdexcept>
#include <string>

int readChoice(const std::string& prompt) {
    int value;
    while (true) {
        std::cout << prompt;
        if (std::cin >> value) {
            std::cin.ignore(std::numeric_limits<std::streamsize
>::max(), '\n'); // Clear the input buffer
            return value;
        }
        if (std::cin.eof()) {
            throw std::runtime_error("Input ended
unexpectedly.");
        }
        std::cout << "Enter a number to navigate the menu." <<
std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::m
ax(), '\n');
    }
}

std::string readLine(const std::string& prompt) {
    std::string s;
    std::cout << prompt;
    std::getline(std::cin, s);
    return s;
}
```

```
void printMenu() {
std::cout << "\n===== Library Menu =====" << std::endl;
std::cout << "1. View all books" << std::endl;
std::cout << "2. View all patrons" << std::endl;
std::cout << "3. Check out a book" << std::endl;
std::cout << "4. Return a book" << std::endl;
std::cout << "5. Add a book" << std::endl;
std::cout << "6. Add a patron" << std::endl;
std::cout << "7. Search by author" << std::endl;
std::cout << "8. Search by genre" << std::endl;
std::cout << "0. Save and exit" << std::endl;
}

int main() {
    Library library;

    try {
        library.loadData();
    } catch (const std::exception& e) {
        std::cerr << "Error loading data: " << e.what() <<
std::endl;
        return 1;
    }

    int choice = -1;
    do {
        printMenu();
        try {
            choice = readChoice("Choice: ");

            switch (choice) {
                case 1:
                    library.displayBooks();
                    break;

                case 2:
```

```
                    library.displayPatrons();
                    break;

                case 3: {
                    int id = readChoice("Patron ID: ");
                    std::string title = readLine("Book title:
");
                    library.checkoutBook(id, title);
                    break;

                }
                case 4: {
                    int id = readChoice("Patron ID: ");
                    std::string title = readLine("Book title:
");
                    library.returnBook(id, title);
                    break;
                }

                case 5: {
                    std::string type = readLine("Type (EBook or
Printed): ");
                    if (type != "EBook" && type != "Printed") {
                        throw std::invalid_argument("Type must
be EBook or Printed.");
                    }
                    std::string title = readLine("Title: ");
                    std::string author = readLine("Author: ");
                    Genre genre = stringToGenre(readLine("Genre
(Fiction, Non-Fiction, Mystery, Science, Biography): "));

                    if (type == "EBook") {
                        double size;
                        std::cout << "File size (MB): ";
                        if (!(std::cin >> size)) {
                            std::cin.clear();
```

```
                            std::cin.ignore(std::numeric_limits
<std::streamsize>::max(), '\n');
                            throw std::invalid_argument("File
size must be a number.");
                        }
                        std::cin.ignore(std::numeric_limits<std
::streamsize>::max(), '\n');
                        library.addBook(new EBook(title,
author, genre, size));
                    }
                    else {
                        int pages = readChoice("Page count: ");
                        library.addBook(new PrintedBook(title,
author, genre, pages));
                    }
                    std::cout << "Added \"" << title << "\"."
<<std::endl;
                    break;
                }

                case 6: {
                    int id = readChoice("New patron ID: ");
                    std::string name = readLine("Name: ");
                    library.addPatron(Patron(name, id));
                    std::cout << "Added patron " << name << "."
<< std::endl;
                    break;
                }

                case 7: {
                    std::string author = readLine("Author: ");
                    library.searchByAuthor(author);
                    break;                }

                case 8: {
```

```
                    Genre genre =
stringToGenre(readLine("Genre: "));
                    library.searchByGenre(genre);
                    break;
                }

                case 0: {
                    library.saveData();
                    std::cout << "Data saved. Goodbye!" <<
std::endl;
                    break;
                }

                default: {
                    std::cout << "Invalid option. Choose 0-8."
<< std::endl;
                }
            }
        } catch (const std::exception& e) {
            std::cerr << "Error: " << e.what() << std::endl;
            if (std::cin.eof()) break;
        }
    } while (choice != 0);

    return 0;
}
```

5 Three Use Case Screen Shots

```
===== Library Menu =====
1. View all books
2. View all patrons
3. Check out a book
4. Return a book
5. Add a book
6. Add a patron
7. Search by author
8. Search by genre
0. Save and exit
Choice: 2
====== Patrons(5) ======
Patron Name: Jane Doe, ID: 1
Borrowed Books: None

Patron Name: John Smith, ID: 2
Borrowed Books: Silent Spring

Patron Name: Maria Garcia, ID: 3
Borrowed Books: None

Patron Name: David Chen, ID: 4
Borrowed Books: None

Patron Name: Emily Eshcoff, ID: 5
Borrowed Books: None
```

```
===== Library Menu =====
1. View all books
2. View all patrons
3. Check out a book
4. Return a book
5. Add a book
6. Add a patron
7. Search by author
8. Search by genre
0. Save and exit
Choice: 3
Patron ID: 5
Book title: Silent Spring
Book 'Silent Spring' checked out by Patron ID 5. Due date: 10/30/2026

===== Library Menu =====
1. View all books
2. View all patrons
3. Check out a book
4. Return a book
5. Add a book
6. Add a patron
7. Search by author
8. Search by genre
0. Save and exit
Choice: 4
Patron ID: 5
Book title: Silent Spring
Book 'Silent Spring' returned by Patron ID 5.
```

```
Choice: 1
====== Books(8) ======
Title: The Great Gatsby
Author: F. Scott Fitzgerald
Genre: Fiction
Status: Available
Number of Pages: 180

Title: Quantum Physics 101
Author: Albert Einstein
Genre: Science
Status: Available
File Size: 4.5 MB

Title: The Hound of the Baskervilles
Author: Arthur Conan Doyle
Genre: Mystery
Status: Available
Number of Pages: 256

Title: Frankenstein
Author: Mary Shelley
Genre: Fiction
Status: Available
File Size: 1.2 MB

Title: The Diary of a Young Girl
Author: Anne Frank
Genre: Biography
Status: Available
Number of Pages: 283

Title: Silent Spring
Author: Rachel Carson
Genre: Non-Fiction
Status: Available
Number of Pages: 368

Title: A Brief History of Time
Author: Stephen Hawking
Genre: Science
Status: Available
File Size: 3.8 MB

Title: And Then There Were None
Author: Agatha Christie
Genre: Mystery
Status: Available
Number of Pages: 272
```

6 GitHub URL

https://github.com/eeshcoff/COMPSCI201